

Pocket Radar: Detailed Software Architecture

Introduction to Practical Electrical Engineering

December 16, 2025

1 System Overview

The Pocket Radar software operates as a multi-threaded Embedded Linux application. It bridges high-level user interaction via a web interface with low-level hardware control via the Linux kernel's SysFS interface. The system is architected around a **Producer-Consumer** model, where the Hardware Thread produces sensor data and the Web Server Thread consumes it for display.

2 Software Block Diagram

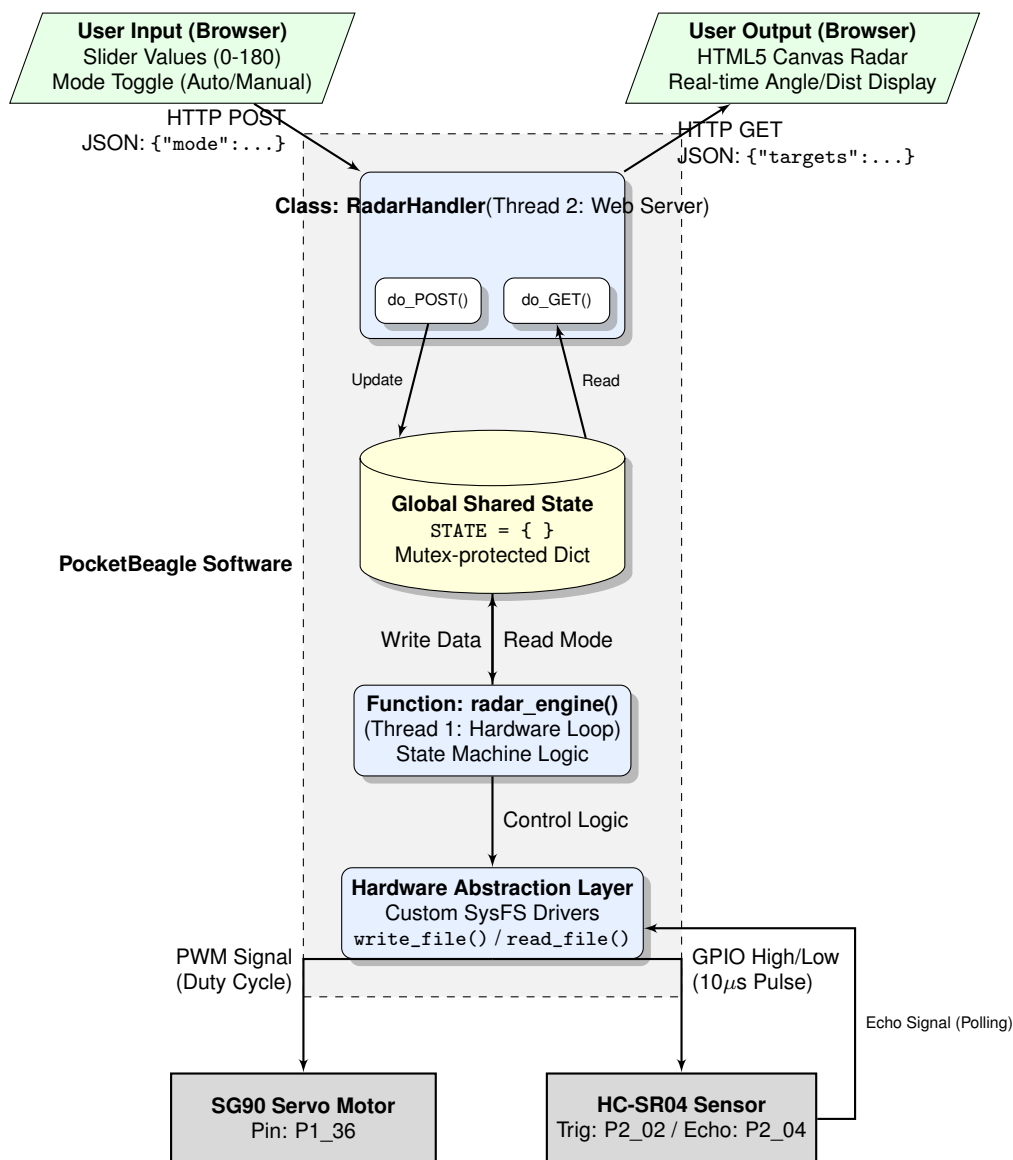


Figure 1: Detailed Data Flow and Thread Interaction Diagram

3 Detailed Component Analysis

3.1 1. Network Interface Layer (Thread 2)

The `RadarHandler` class inherits from Python's `http.server.SimpleHTTPRequestHandler`. It operates asynchronously from the main hardware loop.

- **Input Handling** (`do_POST`): Receives JSON configuration packets from the client.
 - *Example Payload*: `{"mode": "MANUAL", "manual_target": 45}`
 - *Action*: Parses JSON and immediately updates the corresponding keys in the `STATE` dictionary.
- **Output Generation** (`do_GET`): Serves the static HTML interface and dynamic data.
 - *Static*: Returns the HTML/CSS/JS dashboard string on root request (`/`).
 - *Dynamic*: Returns the entire `STATE` dictionary as a JSON string when `/data` is requested. This includes the full `targets` map used for rendering the radar "blips".

3.2 2. Global Shared State (Data Structure)

To ensure thread safety without complex locking mechanisms, the system relies on Python's atomic dictionary operations for a central state object.

```
STATE = {  
    "angle": 90,          # Current physical position of servo (0-180)  
    "distance": 0,        # Most recent instantaneous reading (cm)  
    "mode": "AUTO",       # State Machine Flag: "AUTO" or "MANUAL"  
    "scan_speed": 0.03,   # Delay time in seconds (controls sweep rate)  
    "manual_target": 90,  # Desired angle set by user slider  
    "alarm_threshold": 30, # Safety distance triggering RED alert  
    "targets": {}         # Hash Map: { Angle_Int : Distance_Int }  
}
```

3.3 3. Hardware Control Engine (Thread 1)

The `radar_engine()` function creates an infinite loop that acts as the system's "cortex".

1. **State Check**: Reads `STATE["mode"]` to determine behavior.
2. **Auto Mode Logic**: Increments internal angle counter by step size (3 degrees). Reverses direction (multiplier -1) upon hitting limits (0 or 180).
3. **Manual Mode Logic**: Ignores incrementer. Reads `STATE["manual_target"]` and forces servo to that position.
4. **Sensor Fusion**: Triggers the ultrasonic sensor after movement settles. Updates `STATE["targets"]` with the result at the current angle index.

3.4 4. Hardware Abstraction Layer (HAL)

Due to library incompatibilities with the specific Linux kernel version, standard GPIO libraries were bypassed in favor of raw file system interaction.

- **PWM Control** (`move_servo`): Maps input angle (0-180) to a Duty Cycle (2.5 - 12.5). This signal is generated via the `Adafruit_BBIO.PWM` library on Pin **P1_36**.

- **GPIO Output** (`write_file`): Writes ASCII "1" or "0" to `/sys/class/gpio/gpio59/value` to trigger the sensor (Pin **P2_02**).
- **GPIO Input** (`read_file`): Polls `/sys/class/gpio/gpio58/value` (Pin **P2_04**) to measure the Echo pulse width.